

Tasarım Kuramı: Özgürlüğün Altyapısı Olarak Kısıt

ÖFY

October 2026

Özet

Özgürlük, sınırın yokluğu değil, sınırları tanıyıp onların içinde hareket edebilme kapasitesidir. Sınırları bilmemek seçenekleri çoğaltmaz; görünmeyen sınırlar, kişinin fark edebildiği seçenek sayısını azaltır. Teknik ve disiplin, sınırların içindeki mümkün davranışlar kümesini büyütür ve bu nedenle özgürlüğün karşıtı değil altyapısıdır. Ustalık üç aşamada gelişir: bilgisizlikte sınırsızlık hissi vardır, bilgide gerçek sınırlar görünür olur, ustalıkta ise sınırların arasındaki manevra alanı kullanılır ve gerektiğinde sınırın kendisi değiştirilir. Bale, müzik, yapı mühendisliği, havacılık, yazılım ve dil öğrenimi aynı örüntüyü gösterir: balerin yerçekimini, mühendis fiziği ortadan kaldırmaz; ikisi de sınırın davranışını öğrenerek hareket alanını genişletir. Satranç aynı yapıyı karar düzeyinde gösterir: usta oyuncu daha fazla hamle hesaplamaz, değerlendirmeye değer hamleleri baştan seçer; sınır bilgisi mühendislikte aynı biçimde arama uzayını daraltır. Temsil de bir sınırdır: harita, gösterge paneli ve API gibi araçlar gerçekliğin çoğunu dışarıda bırakır ve tam bu nedenle karar vermeyi mümkün kılar. Sınır aynı zamanda bir müdahale aracıdır: bir sistem akılda tutulamadığında ya da davranışı kestirilemediğinde, sorunun bulunduğu seviyede tek bir sınır sıkılaştırılır ve ardından karmaşıklığın başka bir seviyeye geçip geçmediğine bakılır.

Giriş

Özgürlük gündelik dilde çoğunlukla engelin yokluğu olarak anlaşılır: kural ne kadar azsa, hareket o kadar serbesttir. Bu anlayış siyaset felsefesinde *negatif özgürlük* adıyla bilinir ve bir kişinin başkalarının müdahalesi olmadan hareket edebildiği alanı ifade eder (Berlin, 1958). Beceriye dayalı alanlarda bu tanım yetersiz kalır. Hiç eğitim almamış bir kişi “özgürce dans etmek” istediğinde hiçbir kuralla bağlı değildir, ancak bedeninin yapabildiği hareket sayısı azdır. Yıllarca eğitim almış bir balerin ise çok sayıda kurala uyar ve aynı anda çok daha geniş bir hareket dağarcığına sahiptir. Kurlsız olan daha az şey yapabilmekte, kurallı olan daha fazla şey yapabilmektedir.

Bu durum bir paradoks gibi görünür: disiplin arttıkça özgürlük de artmaktadır. Paradoks, özgürlüğün iki ayrı anlamı birbirine karıştırıldığında ortaya çıkar. Birinci anlamda özgürlük, dışarıdan konan engelin olmamasıdır. İkinci anlamda özgürlük, bir şeyi fiilen yapabilme kapasitesidir. Beceri alanlarında belirleyici olan ikinci anlamdır.

Notta kullanılan merkezî terimler şunlardır:

- **Sınır:** Bir sistemin yapabildikleri ile yapamadıklarını ayıran çizgi. Yerçekimi, malzemenin dayanımı, bir sunucunun saniyede karşılayabildiği istek sayısı birer sınırdır.
- **Teknik:** Sınırların içinde belirli bir sonucu güvenilir biçimde üretmeyi sağlayan öğrenilmiş yöntem.
- **Disiplin:** Tekniğin tekrar yoluyla yerleştirilmesi süreci.
- **Kontrol:** İstenen sonucun istenen anda ve istenen hassasiyetle üretilebilmesi.
- **Özgürlük:** Sınırların içinde kalan seçeneklerden fiilen erişilebilir olanların genişliği.

Bu terimler bir sıra oluşturur: sınırlar öğrenilir, teknik edinilir, disiplinle yerleştirilir, kontrol kazanılır ve özgürlük en sonda ortaya çıkar. Balerin örneğinde sıra şöyle işler: sınırlar öğrenilir, teknik öğrenilir, kaslar kontrol edilir, denge ve müzik öğrenilir, hareket tekrar edilir.

Not iki bölümden oluşur. İlk bölüm kavramsaldır: sınır, arama ve temsil kavramlarını kurar. İkinci bölüm uygulamaya dönüktür: yazılım geliştirmede hangi işaret görüldüğünde hangi sınırın sıkılaştırılacağını bir katalog hâlinde verir.

Notun kapsamı beceri ve tasarım alanlarıyla sınırlıdır. Siyasal özgürlük, hukuki haklar ve toplumsal kısıtlar kapsam dışındadır; burada kurulan çerçeve bu alanlara doğrudan taşınmaz.

Sınır Kavramının Sözlüğü

Sınır tek bir kavram değil, bir kavram ailesidir. Aynı olgunun farklı yönleri farklı terimlerle adlandırılır ve bu terimler birlikte kullanıldığında tasarım düşüncesi için bir dil oluşur.

Sınır ve Kısıt

Sınır sözcüğü geometrik bir çağrışım taşır: bir bölgenin kenarını anlatır. **Kısıt** (*constraint*) ise mühendislikte daha yaygın kullanılan terimdir ve bir çözümün sağlamak zorunda olduğu koşulu ifade eder. Bir köprü için “taşıyabileceği en büyük yük” bir sınırdır; “köprü bu yükü taşımalıdır” ise bir kısıttır. Simon (1969), tasarımı bu tür koşulları sağlayan seçenekler arasında yapılan bir arama olarak tanımlamıştır.

Bir sistem çoğunlukla birden fazla kısıt altında çalışır. Maliyet, performans ve güvenlik aynı anda sağlanması gereken üç ayrı kısıt olabilir ve bunların üçünü birden en üst düzeye çıkarmak çoğu zaman mümkün değildir.

Takas

Takas (*trade-off*), bir özelliği artırmanın başka bir özellikten vazgeçmeyi gerektirdiği durumdur. Performans artırıldığında maliyet de artabilir; güvenlik sıkılaştırıldığında kullanım kolaylığı azalabilir. Takasın olduğu yerde mesele bir sınırı aşmak değil, sınırlar arasında tercih yapmaktır. Kısıtlar birbirinden bağımsız değildir; aralarındaki ilişki, hangi tercihlerin mümkün olduğunu belirler.

Hareket Alanı ve Tasarım Uzayı

Hareket alanı, bir sistemin bütün sınırlarının içinde kalan mümkün davranışlar kümesidir. Özgürlük kavramına en doğrudan bağlanan terim budur: hareket alanı genişledikçe yapılabilecek şeylerin sayısı artar. Daha iyi bir algoritmayla aynı donanım üzerinde daha fazla kullanıcıya hizmet verilmesi, hareket alanının genişletilmesine bir örnektir.

Tasarım uzayı (*design space*), bir problemi çözmek için mümkün olan bütün tasarımların kümesidir. Kısıtlar değiştikçe bu küme de değişir. Yalnızca işlemci (CPU) kullanılabilen bir sistemde çözüm seçenekleri dardır; grafik işlemci (GPU) eklendiğinde daha önce uygulanamayan algoritmalar mümkün hâle gelir; dağıtık mimariye geçildiğinde başka çözümler de açılır. Kısıtlar, tasarım uzayının biçimini belirler.

Serbestlik Derecesi

Serbestlik derecesi (*degree of freedom*), bir sistemin birbirinden bağımsız olarak değişebildiği yönlerin sayısıdır. Uzayda serbestçe hareket eden katı bir cismin altı serbestlik derecesi vardır: üç eksenle öteleme ve üç eksen etrafında

dönme. Bir robot kolunda eklem sayısı arttıkça kolun alabileceği farklı konumların sayısı da artar.

Daha fazla serbestlik derecesi her zaman daha iyi değildir. Bağımsız değişebilen yön sayısı arttıkça sistemin kontrol edilmesi zorlaşır. Bu nedenle özgürlük ile kontrol birlikte gelişmek zorundadır: kontrol edilemeyen bir serbestlik kullanılamaz.

Belirsizlik

Bir sınırın yeri bilinse bile sistemin o sınıra yakın bölgede nasıl davranacağı bilinmeyebilir. **Belirsizlik** bu bilgi eksikliğidir. Belirsizlik altında soru “sınır nerede” olmaktan çıkar ve “sistem hangi koşulda nasıl davranır” hâline gelir. Bu soruya modelleme, benzetim (*simulation*), deney, ölçüm ve geri besleme yoluyla cevap aranır.

Sınırla Yapılan İşlemler

Sınır, üzerinde farklı işlemlerin yapılabildiği bir nesnedir. Aşağıdaki tablo bu işlemleri ve anlamlarını gösterir.

İşlem	Anlamı
Sınır koymak	Sistemin hareket alanını belirlemek
Sınırı bilmek	Gerçekliğin nerede kısıtlandığını anlamak
Sınırı görmek	Daha önce fark edilmeyen bir kısıtı keşfetmek
Sınırı tanımlamak	Belirsiz bir kısıtı açık hâle getirmek
Sınırı sıkılaştırmak	İzin verilen alanı daraltmak
Sınırı gevşetmek	Sisteme daha fazla hareket alanı açmak
Sınırı kaldırmak	Kısıtı tamamen ortadan kaldırmak
Sınırı taşımak	Kısıtın bulunduğu noktayı değiştirmek
Sınırı aşmak	Mevcut kısıtın dışına çıkmak
Sınırı zorlamak	Kısıta çok yakın bir noktada çalışmak
Sınırı ihlal etmek	Sistemin izin vermediği bölgeye girmek
Sınırı genişletmek	Mümkün olanlar kümesini büyütmek
Sınırı daraltmak	Olasılık kümesini küçültmek
Sınırlar arasında hareket etmek	Mevcut kısıtlar içinde çözüm üretmek
Sınırlar arasında denge kurmak	Çelişen kısıtları birlikte gözetmek
Sınırı yeniden tanımlamak	Problemin çerçevesini değiştirmek

Bu işlemler iki gruba ayrılır. Birinci grup sınırı olduğu gibi bırakır ve onun içinde çalışır: bilmek, görmek, zorlamak, arasında hareket etmek. İkinci grup sınırın kendisine müdahale eder: sıkılaştırmak, gevşetmek, taşımak, kaldırmak, yeniden tanımlamak. Uсталık her iki grubu da kapsar.

Kavram Zinciri

Sözlükteki terimler birbirine şu sırayla bağlanır:

sınır → kısıt → ilişki → hareket alanı → tasarım uzayı → serbestlik derecesi → seçenekler → kontrol → özgürlük

Zincirin her halkası bir öncekine dayanır: sınırlar kısıt olarak ifade edilir, kısıtlar arasındaki ilişkiler hareket alanını belirler, hareket alanı tasarım uzayını ve serbestlik derecelerini verir, bunlar seçenekleri üretir, seçenekler kontrol edilebildiği ölçüde özgürlüğe dönüşür.

Bilgisizlikten Ustalığa Üç Aşama

Sınırlarla kurulan ilişki, beceri geliştikçe üç ayrı aşamadan geçer.

Bilgisizlik

Bu aşamada kişi “her şeyi yapabilirim” düşüncesindedir. Bu düşünce bir serbestlik değil, bir bilgi eksikliğidir: neyin mümkün olduğu bilinmediği için neyin mümkün olmadığı da bilinmez. Sınırlar ortadan kalkmış değildir; yalnızca görünmezdir. Görünmeyen bir sınır, görünen bir sınırdan daha kısıtlayıcıdır, çünkü etrafından dolaşmanın yolu aranmaz.

Bir konuyu yüzeysel bilen kişiye “bu nasıl yapılır” diye sorulduğunda aklına bir ya da iki çözüm gelir. Konuyu gerçekten bilen kişi ise aynı işi yapmanın on beş farklı yolunu sayabilir ve hangi koşulda, hangi ölçekte hangisinin uygun olduğunu söyleyebilir. Seçenek azlığı sınırların çokluğundan değil, bilginin azlığından kaynaklanır. Kruger ve Dunning (1999), bir alanda yetersiz olan kişilerin kendi yetersizliklerini fark etmekte de zorlandığını göstermiştir; sınırsızlık hissi bu bulguyla uyumludur.

Bilgi

Bu aşamada kişi “şu kısıtlar var” diyebilir. Gerçek sınırlar görünür hâle gelmiştir. İlk etki çoğunlukla daralma hissidir: daha önce mümkün sanılan pek çok şeyin mümkün olmadığı anlaşılır. Sınırlar bu aşamada engel olarak algılanır.

Ustalık

Bu aşamada kişi “bu kısıtlar arasında şu kadar alanım var; gerekirse şu kısıtı da değiştirebilirim” diyebilir. Sınırlar artık engel değil, tasarımın malzemesidir. Soru “bu kısıt beni neden durduruyor” olmaktan çıkar, “bu kısıt hangi seçenekleri açıyor” hâline gelir.

Ustalığın iki belirgin özelliği vardır. Birincisi, teknik otomatikleşmiştir. Kurallar artık bilinçli olarak akılda tutulması gereken bir yük değildir; beden ve zihin onları kendiliğinden uygular. Dreyfus ve Dreyfus (1986), beceri kazanımını acemilikten uzmanlığa uzanan beş aşamalı bir süreç olarak tanımlamış ve uzmanın kuralları adım adım izlemek yerine durumu sezgisel olarak kavradığını öne sürmüştür. İkincisi, usta aynı problem için çok sayıda çözüm görür ve hangi koşulda hangisinin uygun olduğunu ayırt eder.

Üç aşama şu biçimde özetlenebilir:

Cehalet sınırsızlık hissi verir. Bilgi gerçek sınırları gösterir. Uzmanlık o sınırların içinde manevra alanı yaratır.

Sınırı Bilmek ile Sınırı Kabul Etmek Arasındaki Fark

Bir sınırı bilmek, o sınıra boyun eğmek anlamına gelmez. Bir servisin saniyede 100 istek karşılayabildiği bilindiğinde iki yol açılır:

1. Tasarım bu sınırın içinde kalacak biçimde düzenlenir.
2. Sınırın gereksinimi karşılamadığına karar verilir ve mimari değiştirilerek sınır yerinden oynatılır.

Sınır bilinmediğinde bu iki yolun hiçbirini bilinçli olarak seçilemez; sistemin nerede kırılacağı ancak kırıldığında öğrenilir. Sınır bilgisi hem uyum sağlamanın hem de sınırı değiştirmenin ön koşuludur.

Alanlara Göre Görünüm

Bale

Bir balerinin sınırları arasında bedeninin anatomisi, yerçekimi, denge, kas gücü, esneklik, teknik kurallar, müziğin ritmi, koreografinin mekânı, sahnenin fiziksel boyutları ve partnerin hareketi yer alır.

Engel Olarak Sınır

Yeni başlayan biri için sınırlar doğrudan kapasite eksikliği olarak yaşanır: bacak belirli bir açıdan fazla açılmaz, dönüş tamamlanamaz, denge kaybedilir, yeterince yükseğe sıçranamaz. Bu aşamada dünya ikilidir; bir hareket ya yapılabilir ya yapılamaz.

Teknikle Açılan Seçenekler

Pirouette, tek ayak üzerinde yapılan dönüş hareketidir. Bu hareket dengenin, ağırlık merkezinin, itiş kuvvetinin, bacak pozisyonunun, gövde kontrolünün, zamanlamanın ve *spotting* tekniğinin birlikte yönetilmesini gerektirir. *Spotting*, dönüş sırasında bakışın sabit bir noktada tutulup başın son anda hızla çevrilmesidir ve baş dönmesini azaltır.

Bu bileşenler üzerindeki kontrol arttıkça ikili dünya çözülür ve yerine çok boyutlu bir seçenek kümesi gelir: dönüşün tek mi çift mi olacağı, dönüş sayısı, başlangıç pozisyonu, hız, müzikteki vuruş, yön ve hareketin hangi hareketin devamında yapılacağı ayrı ayrı seçilebilir hâle gelir. Fiziksel sistem aynıdır; değişen, o sistemin içinde erişilebilen bölgenin genişliğidir.

Sınırın Gevşetilmesi

Balerin yalnızca mevcut sınırların içinde daha iyi hareket etmez, bazı sınırları da dışarı taşır. Esneklik buna örnektir: düzenli çalışmayla bir eklem hareket açıklığı zamanla artar; örnek olarak 90° olan bir açıklık önce 120°, sonra 150° olabilir. Bu sayılar yalnızca örnek amaçlıdır. Burada sınır kaldırılmamış, yeri değiştirilmiştir. Yerçekimi aynı kalır, anatomi yok sayılamaz ve kas kapasitesi sonsuz değildir.

Serbestlik Derecesi ve İfade

Teknik kapasite arttıkça aynı bedenle ifade edilebilen hareket sayısı artar. Sıra şöyledir: daha fazla teknik, daha fazla kontrol; daha fazla kontrol, daha fazla mümkün hareket; daha fazla mümkün hareket, daha fazla ifade özgürlüğü.

Görünen Özgürlük

Seyirci sahnede on saniyelik bir hareketi izlediğinde hareketi özgür bulur. Bu izlenimin altında yıllar süren sınır bilgisi ve beden kontrolü vardır. Balerin her şeyi yapamaz; ancak yapabildiklerini çok yüksek hassasiyetle yapabildiği için

dışarıdan sınırsız görünür. Bedenin sınırları ne kadar iyi tanınırsa, o bedenle ne kadar farklı şey yapılabileceği o kadar iyi keşfedilir.

Bale ile yazılım arasındaki koşutluk şudur: yazılım mühendisi kısıtları bildiği için sistemin tasarım uzayını görür; balerin bedeninin kısıtlarını bildiği için bedeninin ifade uzayını genişletir.

Müzik

Hiç nota bilmeyen biri piyanoda “özgürce çalmak” istediğinde yapabilecekleri sınırlıdır. Akorlar, gamlar, ritim ve armoni öğrenildikten sonra kurallar üzerinde düşünmeden doğaçlama yapılabilir. Caz bunun belirgin bir örneğidir: doğaçlama, kuralların yokluğunda değil, kuralların otomatikleştiği noktada mümkün olur. Stravinsky (1947) benzer bir görüşü besteci açısından savunmuş ve kendine koyduğu kısıtlar artıkça daha serbest çalıştığını belirtmiştir.

Yapı ve Havacılık Mühendisliği

Köprü

Bir köprü tasarımında “istenen her şeyin çizilebilmesi” anlamında bir özgürlük yoktur. Gerçek dünya sürekli sınır koyar: malzemenin dayanımı, köprünün kendi ağırlığı, yerçekimi, rüzgâr, trafik yükü, sıcaklık değişimi, yorulma (*fatigue*; tekrarlanan yük altında malzemenin zamanla zayıflaması), üretim yöntemleri, bütçe, güvenlik standartları ve geometrik kısıtlar. Bu sınırlar yok sayıldığında tasarımcı özgürleşmez, köprü yıkılır.

Deneyimsiz bir mühendis problemi “buraya nasıl bir köprü konabilir” biçiminde kurar. Deneyimli bir mühendis ise açıklığı, yükü, kullanılabilecek malzemeleri ve izin verilen yer değiştirme miktarını sayar, ardından bu koşullar altında hangi yapı tiplerinin mümkün olduğunu sorar. İkinci mühendis daha fazla kısıtı hesaba katmakta ve buna rağmen daha fazla seçenek görmektedir, çünkü kısıtların nasıl davrandığını bilmektedir.

Uçak

Bir havacılık mühendisine “uçacağı istediğin gibi tasarla” denmesi serbestlik sağlamaz. Uçak uçmak, belirli bir hızda seyretmek, belirli bir yük taşımak, kabul edilebilir miktarda yakıt tüketmek, kanatları sağlam kalmak, motoru çalışmak ve titreşimleri kontrol altında tutmak zorundadır. Bunların her biri bir sınırdır.

Aerodinamik, malzeme bilimi, kontrol teorisi ve yapısal mekanik öğrenildikçe bu sınırlar içindeki tasarım uzayı büyür. Kanat biçimi ve açıklığı değiştirilebilir, kompozit malzeme kullanılarak ağırlık azaltılabilir, aktif kontrol sistemleri eklenebilir. Ulaşılan özgürlük “her şeyi yapabilmek” değil, “bu gerçeklik içinde çok daha fazla şeyi mümkün kılabilme”tir.

Yaratıcılığın Kaynağı

İyi bir mühendisin yaratıcı görünen çözümleri çoğu zaman kısıtları kaldırmaktan değil, kısıtların yapısını anlamaktan doğar. Kısıtların birbiriyle ilişkisi kavrandığında, bu ilişkiler kullanılarak daha önce düşünülmemiş bir çözüm bölgesi açılır.

Yazılım

Dosya İşleme Sistemi Örneği

Gereksinim şöyle olsun: kullanıcı bir ses dosyası yükler, sistem dosyayı işler, kullanıcı sonucu indirir. Gereksinim basit görünür, ancak gerçek sistem sınırlarla doludur:

- Dosya 2 GB büyüklüğünde olabilir.
- Yükleme sırasında ağ bağlantısı kopabilir.
- Aynı dosya iki kez yüklenebilir.
- İşlem 30 dakika sürebilir.
- Kullanıcı tarayıcıyı kapatabilir.
- İki **worker** (işleri arka planda yürüten süreç) aynı işi almaya çalışabilir.
- Disk dolabilir.
- Servis çöktüğünde işlem yarıda kalabilir.
- Kullanıcının yetkisi olmayabilir.
- Veritabanına erişilemeyebilir.

Deneyimsiz yaklaşım yalnızca yükleme uç noktasını (*endpoint*) yazar. Kod çalışır, ancak sistemin küçük bir bölümü çözülmüştür.

Deneyimli yaklaşım önce sistemin durumlarını sorar. Bir iş sırasıyla **PENDING**, **PROCESSING**, **COMPLETED** ve **DOWNLOADED** durumlarından geçer. Hata olduğunda **PROCESSING** durumundan **FAILED** durumuna, oradan **RETRYING** durumuna ve yeniden **PROCESSING** durumuna döner. Durumlar ve aralarındaki geçişler yazıldığında sistemin *durum uzayı*, yani içinde bulunabileceği bütün durumların kümesi görünür olur.

Her kısıt belirli bir tekniğe götürür. Bu eşleşmeler dağıtık sistemler literatüründe yerleşiktir (Kleppmann, 2017).

Kısıt	Açtığı teknik
Aynı iş iki kez çalışmamalı	<i>Idempotency</i> : bir işlemin birden fazla kez uygulanmasının tek uygulamayla aynı sonucu vermesi
Worker çökerse iş kaybolmamalı	Kuyruk ve kalıcı durum kaydı
Kullanıcı sonucu beklemek zorunda kalmamalı	Eşzamansız işleme ve bildirim
İki worker aynı işi almamalı	Kilitleme ya da işi sahiplenme mekanizması
Dosya çok büyük	Akış hâlinde okuma (<i>streaming</i>) ve parçalı yükleme (<i>multipart upload</i>)

Zincir şu sırayla işler: kısıt fark edilir, kavranır, bir tekniğe bağlanır ve teknik yeni tasarım seçenekleri açar. Bu süreçte problemin tanımı da değişir. Başlangıçta “dosya yükleyen bir arayüz” olarak görülen iş, kısıtlar anlaşıldıkça “güvenilir bir dağıtık işleme sistemi” olarak görülmeye başlanır. Soyutlama bu noktada ortaya çıkar.

Düğme Örneği

Bir düğmeye basıldığında sunucuya istek gönderilsin. İlk kısıt, kullanıcının düğmeye art arda on beş kez basabilmesidir. Basit çözüm her basışta istek gönderir. Bir adım ileri, gönderim sürerken gelen basışları yok sayar.

Ardından başka kısıtlar görünür: istek zaman aşımına uğrayabilir, arayüz bileşeni cevap gelmeden ekrandan kaldırılabilir, gelen cevap eskimiş olabilir, kullanıcı sayfayı değiştirmiş olabilir, aynı kaynak başka bir yerden güncellenmiş olabilir. Bu kısıtlar sırasıyla durum yönetimi, iptal, eşzamanlılık ve tutarlılık kavramlarına götürür. Tek bir düğme, yeterince dikkatli bakıldığında yazılım mühendisliğinin temel konularının çoğunu içerir.

Yazılımda Ustalık Ölçütü

İyi yazılım mühendisi daha fazla kod yazabilen kişi değil, sistemin hareket edebileceği alanı daha doğru görebilen kişidir. Yazılım mühendisliği büyük ölçüde “bu sistemin gerçek sınırı nerede” sorusunun tekrar tekrar sorulmasıdır. Başlangıçta kısıtlar engel olarak görülür; ustalıkla birlikte her kısıtın hangi tasarım seçeneklerini açtığı sorulmaya başlanır.

Dil

Dil öğrenimi aynı örüntüyü en yalın hâliyle gösterir. Az sözcük bilen kişinin söyleyebileceği şeyler azdır. Dilbilgisini bilmeyen kişinin kurabileceği yapılar azdır. Dil gerçekten öğrenildiğinde aynı düşünce çok sayıda farklı biçimde ifade edilebilir. Dilbilgisi kuralları ifadeyi kısıtlamaz; ifadenin mümkün biçimlerini üretir.

Satranç ve Karar Sistemi Olarak Mühendislik

Satranç, küçük ve kuralları tam olarak bilinen bir karar verme sistemidir: bir konum vardır, o konumdan yapılabilecek hamleler vardır, her hamle yeni bir konum üretir. Mühendislik kararları aynı yapıyı taşır. Bu nedenle satrancın dört kavramı, durum uzayı, arama uzayı, strateji ve optimizasyon, mühendislik düşüncesine doğrudan aktarılabilir.

Durum Uzayı

Durum (*state*), bir sistemin belirli bir andaki tam tarifidir. Satrançta durum, tahtanın o andaki konumudur: hangi taşın hangi karede olduğu ve sıranın kimde olduğu. **Durum uzayı** (*state space*) ise sistemin bulunabileceği bütün olası durumların kümesidir.

Bir yazılım sisteminde durum ölçülebilir büyüklüklerle tarif edilir: işlemci kullanımı %70, kuyrukta bekleyen iş sayısı 340, açık veritabanı bağlantısı 100 üzerinden 80, çalışan **worker** sayısı 5. Bu dört sayı birlikte sistemin o andaki konumunu verir.

Durumlar daha kaba bir düzeyde de adlandırılabilir. Bir web sistemi normal çalışmadan yüksek yüke, yüksek yükten kuyruğun dolmasına, oradan bir **worker** arızasına, ardından kurtarma sürecine ve yeniden normal çalışmaya geçebilir. Bu adların her biri ayrı bir durumdur ve aralarındaki geçişler sistemin davranışını tarif eder.

Durum uzayını yazmak, “sistem şu anda nerede” sorusuna cevap verebilmenin ön koşuludur. Durumları adlandırılmamış bir sistemde arıza, hangi durumdan hangi duruma geçildiği bilinmeden yaşanır.

Arama Uzayı

Arama uzayı (*search space*), belirli bir durumdan değerlendirilebilecek seçeneklerin kümesidir. Satrançta açılış konumunda beyaz, şah piyonunu iki kare sürebilir (e4), atını geliştirebilir (Nf3) ya da vezir kanadı piyonunu sürebilir (c4). Bunlar ve diğer geçerli hamleler birlikte o konumun arama uzayını oluşturur.

Mühendislikte hamlelerin karşılığı sistemde yapılabilecek değişikliklerdir: önbellek (*cache*) eklemek, kuyruk koymak, veritabanını ölçeklemek, işleri toplu hâlde işlemek (*batching*), eşzamanlı çalışan iş sayısını azaltmak. Her biri sistemi başka bir duruma taşıyan bir hamledir.

Durum uzayı ile arama uzayı farklı sorulara cevap verir. Durum uzayı sistemin *nerede olabileceğini*, arama uzayı ise oradan *nereye gidilebileceğini* gösterir.

Strateji ve Taktik

Satrançta hamleler tek tek, birbirinden kopuk değerlendirilmez. Oyuncu “şimdi kaleyi oynayayım” diye düşünmez; merkezi kontrol etmeyi, taşlarını geliştirmeyi ve ardından rakip şaha baskı kurmayı içeren bir plan izler.

- **Taktik:** Belirli bir problemi çözen yakın vadeli hamle ya da kısa hamle dizisi.
- **Strateji:** Sistemin hangi yönde değiştirileceğine dair uzun vadeli yaklaşım.

Mühendislikte “bu uç noktayı hızlandırayım” kararı taktiktir. “Önce ölçüm yapılacak, darboğaz belirlenecek, sistem değiştirilecek ve yeniden ölçülecek” biçimindeki sıra ise stratejidir. Strateji olmadan yapılan taktik hamleler yerel olarak doğru olabilir ve bütünde bir yere varmayabilir; ölçmeden yapılan bir hızlandırma, darboğaz olmayan bir bölümü iyileştirir.

Optimizasyon

Amaç Fonksiyonu ve Kısıtlar

Optimizasyon, kısıtları sağlayan seçenekler arasından belirli bir ölçüte göre en iyisinin seçilmesidir. Ölçüt, **amaç fonksiyonu** (*objective function*) adını alır. Genel biçim şöyledir (Boyd ve Vandenberghe, 2004):

$$\min_x f(x) \quad \text{kosul : } g_i(x) \leq 0, \quad i = 1, \dots, m$$

Burada x bir tasarımı, f en aza indirilmek istenen büyüklüğü, g_i ise kısıtları gösterir. Bir yazılım sisteminde bu şu biçimi alabilir: gecikme süresi (*latency*) en aza indirilecektir, ancak aylık maliyet 1000 \$’ı geçmemeli, erişilebilirlik %99,9’un altına düşmemeli ve bellek kullanımı 16 GB’ı aşmamalıdır.

Süreç iki adımlıdır. Önce bütün çözümler kısıtlardan geçirilir ve kısıtları sağlayanlar ayrılır; bu kümeye *uygun çözümler* (*feasible set*) denir. Ardından uygun çözümler arasından amaç fonksiyonuna göre seçim yapılır. Kısıtlar burada elemeyi, amaç fonksiyonu sıralamayı yapar.

Optimizasyon ve Takas

Optimizasyon her şeyi en üst düzeye çıkarmak değildir. Bir büyüklük iyileştirilirken çoğunlukla bir başkasından vazgeçilir:

- Daha hızlı sistem, daha yüksek maliyet getirir.
- Daha güvenli sistem, daha fazla gecikme getirir.
- Daha basit mimari, daha az esneklik getirir.

Bu nedenle optimizasyonun sonucu tek bir “en iyi” değil, belirli bir tercihe göre en iyidir. Tercih değiştiğinde seçilen çözüm de değişir.

Ortak Döngü

Satranç ve mühendislik aynı döngüyü izler. Aşağıdaki tablo adımları karşılaştırır.

Adım	Satranç	Mühendislik
Durum	Tahtadaki konum	Sistemin ölçülen durumu
Seçenekler	Geçerli hamleler	Yapılabilecek değişiklikler
Arama	Hamle dizilerinin hesaplanması	Tasarım uzayının taranması
Değerlendirme	Konumun gücünün tartılması	Ölçütlere göre karşılaştırma
Seçim	Hamlenin oynanması	Kararın uygulanması
Yeni durum	Yeni konum	Sistemin yeni durumu

Bir ses işleme sisteminde kuyrukta 10.000 dosyanın biriktiği durum ele alınabilir. Yapılabilecek hamleler şunlardır: **worker** sayısını artırmak, işleri toplu hâlde işlemek, kuyruk altyapısını değiştirmek, eşzamanlılığı artırmak, önbellek eklemek. Her hamle dört ölçüte göre değerlendirilir: gecikme süresi, maliyet, güvenilirlik ve karmaşıklık. Seçilen hamle uygulandığında sistem yeni bir duruma geçer ve döngü baştan başlar.

Aramanın Daraltılması

Satrançta bütün olası hamle dizilerini hesaplamak mümkün değildir. Shannon (1950), olası satranç oyunlarının sayısını 10^{120} mertebesinde tahmin etmiş ve bu nedenle bütün hamleleri tarayan bir yaklaşım yerine yalnızca makul hamleleri inceleyen seçici bir aramanın gerektiğini belirtmiştir.

İnsan oyuncular için de aynı durum geçerlidir. De Groot (1965), ustaların zayıf oyunculardan belirgin biçimde daha fazla hamle hesaplamadığını, ancak baştan daha iyi hamleleri değerlendirmeye aldığını bulmuştur. Chase ve Simon (1973) bu farkı algıya bağlamıştır: ustalar konumu tek tek taşlar olarak değil, anlamlı taş grupları olarak görür.

İyi oyuncu çok hamle bilen kişi değil, hangi hamlelerin değerlendirmeye değer olduğunu bilen kişidir. Mühendislikte de her mümkün çözümü denemek imkânsızdır. Beceri, çok büyük bir çözüm uzayını anlamlı bir alt kümeyle indirip aramayı orada yapabilmektir.

Sınır bilgisi bu daraltmayı sağlayan şeydir. Kısıtlar, arama uzayının büyük bölümünü denemeye gerek kalmadan eler; geriye kalan bölge hem daha küçük hem de daha verimlidir. Böylece sınır iki ayrı iş görür: mümkün olanı gösterir ve bakılmaya değmeyeni ayıklar.

Bu sıra soyutlama ile birleşir. Gerçek sistemden bir model çıkarılır, modelden durum uzayı elde edilir, durum uzayından arama uzayı türetilir, seçenekler değerlendirilir ve seçim yapılır. Modelin kalitesi, sonraki bütün adımların kalitesini belirler.

Temsil ve Bilgi Yoğunluğu

Temsil (*representation*), bir şeyin yerine geçen ve onun hakkında düşünmeyi sağlayan daha küçük bir nesnedir: bir harita, bir sayı, bir çizim, bir kod satırı. **Bilgi yoğunluğu**, küçük bir temsilin ne kadar çok şey anlatabildiğinin ölçüsüdür. Az sayıda işaretle çok anlam taşıyan temsil yoğundur; çok sayıda işaretle az anlam taşıyan temsil seyrekir.

Buradaki “bilgi” sözcüğü, iletişim kuramındaki teknik anlamından farklıdır. Shannon (1948) bilgiyi, bir iletinin giderdiği belirsizlik miktarı olarak ve iletinin anlamından bağımsız biçimde tanımlamıştır. Bu notta kastedilen ise karar vermeye yarayan anlamdır: bir temsilin yoğunluğu, ona bakan kişinin ne kadar isabetli karar verebildiğiyle ölçülür.

Harita

Bir şehir gerçek boyutuyla bir masaya konamaz. Şehrin haritası ise birkaç çizgi ve noktadan oluşur. Bu çizgilerden yollar, kavşaklar, mesafeler, yönler ve bölgelerin birbirine göre konumu okunabilir. Görsel malzeme azdır, taşınan anlam çoktur.

Harita, şehrin kendisi değildir ve olması da istenmez. Korzybski (1933) bu ayrımı “harita arazi değildir” biçiminde ifade etmiştir. Haritanın değeri, araziye eksiksiz kopyalamasından değil, yol bulmak için gereken bilgiyi tutup geri kalanını atmasından gelir. Binaların rengini de gösteren bir harita daha fazla veri içerir ve yol bulmayı zorlaştırır.

Günlük Kayıtları ve Gösterge Paneli

Bir yazılım sistemi çalışırken yaptığı her işi satır satır kaydeder; bu kayıtlara *log* denir. On bin satırlık bir kayıt dosyasında isteğin başladığı, bağlantının açıldığı, sorgunun çalıştığı tek tek yazılıdır. Sistemin genel durumu bu satırların içindedir, ancak okunarak çıkarılması uzun sürer.

Bir gösterge paneli (*dashboard*) aynı sistemi beş sayıyla gösterebilir:

Ölçüm	Değer
İstek sayısı	12.430 / dakika
Gecikme süresi	180 ms
Hata oranı	%0,8
Kuyruktaki iş	4.200
Worker sayısı	12

Bu beş sayı, sistemin o andaki durumu hakkında on bin satırdan daha hızlı ve daha kullanışlı bilgi verir. Panel, kayıt dosyasından daha yoğun bir temsildir ve sistemin durumunu doğrudan okunabilir hâle getirir.

Veri Çokluğu ile Anlam Yoğunluğu Arasındaki Fark

Bilgi yoğunluğu, küçük bir alana çok veri sıkıştırmak demek değildir. Bir panele işlemci kullanımı %73,4821, bellek kullanımı %61,2847, disk kullanımı %48,1937 gibi yüz ayrı sayı konabilir. Veri miktarı artmıştır, ancak sistemde ne olduğu anlaşılmaz hâle gelmiştir. Veri çoktur, anlam yoğunluğu düşüktür.

Yoğunluğu belirleyen, işaret başına düşen veri değil, işaret başına düşen *işe yarar* anlamdır. Tufte (1983) aynı ilkeyi grafik tasarımı için ortaya koymuş ve bir grafikteki mürekkebin mümkün olan en büyük bölümünün veriyi göstermeye ayrılmasını, veri taşımayan her ögenin çıkarılmasını önermiştir.

Çizim

Bir ressamın bir yüzü fotoğraf gibi ayrıntılı çizmesi gerekmez. İki göz ve bir ağız için atılmış birkaç çizgi, bakan kişide “bu bir yüz” algısını oluşturmaya yeter. Çizgi sayısı azalmış, temel anlam korunmuştur.

Soyutlama ile İlişkisi

Soyutlama ile bilgi yoğunluğu aynı işlemin iki yüzüdür:

- **Soyutlama**, gereksiz ayrıntıların çıkarılmasıdır.
- **Bilgi yoğunluğu**, geriye kalan temsilin mümkün olduğunca fazla anlam taşımasıdır.

Soyutlama neyin atılacağına, yoğunluk ise kalanın ne kadar iş gördüğüne bakar. Yanlış ayrıntı atıldığında temsil küçülür ve aynı anda anlamını da kaybeder.

API Örneği

API (*Application Programming Interface*; uygulama programlama arayüzü), bir yazılımın başka yazılımlara sunduğu çağrı biçimlerinin bütünüdür. Web üzerinde yaygın olan REST tarzında (Fielding, 2000) bir çağrı şöyle yazılır:

GET /users/42

On üç karakterlik bu ifade dört ayrı şey söyler:

- GET: Veri istenmektedir; sistemde bir şey değiştirilmeyecektir.
- /users: İstenen şey kullanıcı kaynağıdır.
- 42: Kullanıcılardan belirli bir tanesi kastedilmektedir.
- İfadenin biçimi: Etkileşim HTTP protokolü üzerinden yürümektedir.

Arkada çalışan veritabanı sorgusu, yetki denetimi, ağ bağlantısı ve veri biçimi dönüşümü ifadede yer almaz. Temsil, karar için gerekeni tutmuş ve geri kalanını gizlemiştir.

Mühendislikteki Temsil Araçları

Mühendislikte değer verilen araçların çoğu yoğun temsillerdir:

- **Diyagram:** Bir sistemin parçalarını ve aralarındaki ilişkileri gösterir, iç işleyişi göstermez.
- **Ölçüm** (*metric*): Çok sayıda olayı tek bir sayıya indirger.
- **Tip** (*type*): Bir değer hangi biçimde olabileceğini ve onunla ne yapılabileceğini tek sözcükle bildirir.
- **Model:** Gerçek sistemin, belirli bir soruyu cevaplamaya yetecek kadar sadeleştirilmiş hâlidir.
- **Gösterim** (*notation*): Uzun bir sözel tarifi kısa bir simge dizisine çevirir.

Bunların hiçbirisi gerçek sistemin tamamını taşımaz; işe yarayan bölümünü sıkıştırılmış biçimde taşır. Box (1976) aynı düşüncüyü modeller için dile getirmiş, bütün modellerin yanlış olduğunu, bazılarının ise yararlı olduğunu belirtmiştir.

İyi bir temsil, gerçekliğin bütün ayrıntılarını taşımaz; karar vermek için gereken bilgiyi yoğunlaştırır.

Sınır ve Arama ile Bağlantısı

Temsil, bilinçli olarak konmuş bir sınırdır. Beş sayılık bir panel, dikkatin beş şeye yöneltilmesini ve geri kalanının dışarıda bırakılmasını sağlar. Bu sınır dikkati kısıtlar ve aynı anda kararı mümkün kılar; on bin satırı okuyan kişi karar veremez, beş sayıya bakan kişi verebilir.

Ustaların satranç konumunu anlamlı gruplar olarak görmesi, zihinde daha yoğun bir temsil taşımaları anlamına gelir. Yoğun temsil, arama uzayını daraltan araçtır: iyi kurulmuş bir temsil, değerlendirmeye değer seçenekleri öne çıkarır ve diğerlerini görüş alanından çıkarır.

Müdahale Aracı Olarak Sınır

Çıkış Gözlemi

Kod kararsız hissedildiğinde dosyaların 200 satır sınırına çekilmesi, kararlılık hissini geri getirir. Bu müdahale kod seviyesindedir. Sınırın işe yaramasının nedeni, dosyayı yeniden akılda tutulabilecek boyuta indirmesidir. İnsanın aynı anda akılda tutabildiği öge sayısı sınırlıdır (Miller, 1956); bir parça bu sınırı aştığında davranışı kestirilemez hâle gelir.

Genel Kural

Bir şey akılda tutulamadığında ya da davranışı önceden kestirilemediğinde, o seviyede bir sınırı sıkılaştırma zamanı gelmiştir. Aynı mantık kodun dışındaki seviyelerde de geçerlidir.

Müdahale Zamanını Gösteren İşaretler

- Bir değişikliğin neyi etkileyeceği önceden söylenememektedir.
- Aynı türden hata tekrar tekrar ortaya çıkmaktadır.
- Bir bölüme dokunmaktan çekinilmektedir.
- Sistemin o parçası birkaç cümleyle anlatılamamaktadır.
- Sürprizler artmakta, “bu nasıl oldu” sorusu sıklaşmaktadır.

Seviyelere Göre Sınırlar

Seviye	İşaret	Sıkılaştırılacak sınır
Değişiklik boyutu	İncelemede hatalar kaçmakta, geri almak zorlaşmaktadır	<i>Commit</i> ve PR (<i>pull request</i> ; birleştirme isteği) boyutu
Bağımlılıklar	Bir yer değiştirildiğinde beş yer bozulmaktadır	Modülün dışarıya açtığı yüzey ve kimin kimi çağırabildiği
Durum	“Sistem bu hâle nasıl geldi” türü hatalar	Durum sayısı ve izin verilen geçişler
Veri	Beklenmeyen değerler gelmektedir	Tipler, veritabanı kısıtları, girişte doğrulama
Çalışma zamanı	Kuyruk şişmekte, zaman aşımaları artmaktadır	Eşzamanlılık, kuyruk boyu, zaman aşımı, istek hızı
Yayın	Büyük sürümler bir şeyleri bozmaktadır	Sürüm boyutu: küçük ve sık yayın
Yarım işler	Aynı anda çok iş açıktır, hiçbiri bitmemektedir	Aynı anda yürüyen iş sayısı
Kapsam	İş uzadıkça uzamaktadır	O turda yapılacakların listesi
İzleme	Çok alarm vardır, hangisinin önemli olduğu belli değildir	Alarm ve ölçüm sayısı

Karmaşıklığın Yer Değiřtirmesi

Sınır sıkılařtırmak karmaşıklığı her zaman yok etmez, bazen yalnızca yerini deęiřtirir. Brooks (1987), yazılımdaki karmaşıklığın bir bölümünün problemin doęasından geldiğini ve hiçbir araçla ortadan kaldırılamadığını savunmuřtur. 200 satır sınırı dosyayı sadeleřtirir, ancak dosya sayısı artar ve karmaşıklık dosyaların arasına geçebilir. Bu durumda bir sonraki müdahale kod seviyesinde deęil, baęımlılık seviyesinde yapılır.

Sıra řöyle işler: dengesizliğin hissedildięi seviye bulunur, o seviyede tek bir sınır sıkılařtırılır, ardından karmaşıklığın başka bir seviyeye kaçıp kaçmadığına bakılır.

Katalogun Okunması

İzleyen bölümlerdeki tablolar aynı biçimi tařır: sol sütunda gözlenen işareti, saę sütunda yapılacak müdahale yer alır. Tablolardaki sayılar (200 satır, dört parametre, 300–400 satırlık deęişiklik, üç açık iş, 30 dakika) başlangıç deęerleridir; kesin eřik deęildir.

Katalog: Kod ve Tasarım

Kod

İşaret	Müdahale
Dosya 200 satırı geçmiştir	Dosya bölünür; her dosya tek bir iş yapar.
Fonksiyon ekrana sığmamaktadır	Parçalanır; her parçaya ne yaptığını söyleyen bir ad verilir.
Fonksiyon dörtten fazla parametre almaktadır	Parametreler bir yapıda toplanır ya da fonksiyon ikiye ayrılır.
İç içe üçten fazla if vardır	Erken dönüş kullanılır; koşullar ayrı fonksiyona taşınır.
Aynı kod üçüncü kez yazılmaktadır	Ortak fonksiyona çıkarılır. İkinci tekrarda beklenir, üçüncüde çıkarılır (Fowler, 1999).

İsimlendirme ve Okunabilirlik

İşaret	Müdahale
Bir değişkenin ne tuttuğunu anlamak için kodun okunması gerekmektedir	İsim değiştirilir.
Kodu açıklamak için yorum yazılmaktadır	Önce kod sadeleştirilir. Yorum “neden” sorusunu cevaplar, “ne” sorusunu değil.
Aynı kavram için üç farklı isim kullanılmaktadır	Biri seçilir ve her yerde o kullanılır.
Kodda anlamı belli olmayan sayılar vardır	İsimli sabite çevrilir.

Soyutlama

İşaret	Müdahale
Soyutlamayı kullanmak için içinin okunması gerekmektedir	Soyutlama sızdırmaktadır. Arayüz düzeltilir ya da soyutlama kaldırılır.
Tek bir kullanımı olan genel bir yapı vardır	Sadeleştirilir. İkinci kullanım geldiğinde genelleştirilir.
Yeni bir özellik için beş katmana dokunmak gerekmektedir	Katman sayısı azaltılır.
Bir fonksiyon <code>boolean</code> parametreyle iki farklı iş yapmaktadır	İki ayrı fonksiyona bölünür.

Bağımlılıklar

Bir modülün iç ayrıntılarını gizleyip dışarıya yalnızca küçük bir yüzey açması ilkesi Parnas (1972) tarafından ortaya konmuştur.

İşaret	Müdahale
Bir dosya değiştirildiğinde başka yerler bozulmaktadır	Modülün dışarıya açtığı ögeler azaltılır; geri kalanı gizlenir.
A modülü B'yi, B modülü de A'yı çağırılmaktadır	Döngü kırılır. Ortak kısım üçüncü bir modüle taşınır.
Bir modülü test etmek için beş şeyin ayağa kaldırılması gerekmektedir	Bağımlılıklar parametre olarak verilir.

Mimari

İşaret	Müdahale
Her yeni özellik aynı büyük dosyaya eklenmektedir	Dosya sorumluluklarına göre ayrılır.
Bir servis diğerleri olmadan çalıştırılmamaktadır	Aradaki sınır yanlıştır. Ya birleştirilir ya da arayüz netleştirilir.
İki modül sürekli birlikte değişmektedir	Birleştirilir. Birlikte değişen şeyler birlikte durur.
Bir modülün içindeki parçalar hiç birlikte değişmemektedir	Ayrılır.

API Tasarımı

İşaret	Müdahale
İstemci bir iş için beş ayrı çağrı yapmaktadır	O iş için tek bir uç nokta açılır.
Bir alanın değiştirilmesi istemcileri bozmaktadır	Alan silinmez, yeni alan eklenir. Eskisi kullanım bitince kaldırılır.
Uç nokta her şeyi döndürmektedir	Sayfalama ve alan seçimi eklenir. Varsayılan yanıt küçük tutulur.
Aynı istek iki kez geldiğinde iki kayıt oluşmaktadır	İsteğe benzersiz anahtar eklenir; tekrarı tanınır ve yok sayılır.
Hata yanıtları her uç noktada farklı biçimdedir	Tek bir hata biçimi belirlenir ve hepsinde kullanılır.

Geriye Uyumluluk

İşaret	Müdahale
Eski istemciler hâlâ kullanımdadır	Kaldırmadan önce kullanım ölçülür. Sıfıra indiğinde kaldırılır.
Davranış değişikliği sessizce yapılmıştır	Değişiklik günlüğü tutulur. Kıran değişiklik önceden duyurulur.
İki sürümü aynı anda desteklemek kodu karıştırmaktadır	Eski sürüm için son tarih konur ve uygulanır.

Katalog: Durum ve Veri

Durum

İşaret	Müdahale
“Bu kayıt bu hâle nasıl geldi” sorusu sorulmaktadır	Durumlar tek tek yazılır. İzin verilen geçişler listelenir, geri kalanı kodda yasaklanır.
Aynı bilgi iki yerde tutulmaktadır	Biri silinir. Tek kaynak kalır, diğeri ondan hesaplanır.
Durum birkaç <code>boolean</code> ile tutulmaktadır	Tek bir durum alanına çevrilir (<code>PENDING</code> , <code>PROCESSING</code> , <code>FAILED</code> gibi).

Veri

İşaret	Müdahale
Beklenmeyen değer gelmiştir	Girişte doğrulanır. İçeri bozuk veri alınmaz.
Veritabanında olmaması gereken bir kayıt çıkmıştır	Kısıt eklenir: <code>NOT NULL</code> , <code>UNIQUE</code> , <code>FOREIGN KEY</code> , <code>CHECK</code> .
Aynı hata türü ikinci kez veriden kaynaklanmıştır	Tip daraltılır; o değer kurulamaz hâle getirilir.

Veritabanı ve Şema

İşaret	Müdahale
Şema değişikliği yayını kırmıştır	Üç adıma bölünür: önce eski kodla uyumlu değişiklik, sonra kod, en son temizlik.
Sorgu veri büyüdükçe yavaşlamıştır	Sorgu planına bakılır, eksik indeks eklenir.
Tabloda ne işe yaradığı bilinmeyen sütunlar vardır	Kullanım ölçülür, kullanılmayan kaldırılır.
Büyük bir güncelleme tabloyu kilitlemiştir	Güncelleme küçük gruplara bölünür.
Silinen veri sonradan gerekli olmuştur	Kritik tablolarda silme yerine işaretleme kullanılır.

Sayı ve Para

İkili kayan noktalı sayılar ondalık kesirlerin çoğunu tam olarak gösteremez (Goldberg, 1991); kuruş farkları bu nedenle ortaya çıkar.

İşaret	Müdahale
Kuruş farkları çıkmaktadır	Para ondalıklı sayı olarak tutulmaz. En küçük birimde tam sayı kullanılır.
Toplamlar raporlar arasında tutmamaktadır	Yuvarlama tek bir yerde ve en sonda yapılır.
Sayaç taşmakta ya da eksiye düşmektedir	Sınır kontrolü eklenir. Veritabanına CHECK kısıtı konur.
Birimler karışmaktadır (saniye/milisaniye, bayt/kilobayt)	Birim değişken adına yazılır.

Metin ve Kodlama

İşaret	Müdahale
Türkçe karakterler bozuk çıkmaktadır	Her katmanda UTF-8 kullanılır: dosya, veritabanı, bağlantı, yanıt başlığı.
Büyük/küçük harf karşılaştırması	Dile bağlı olmayan karşılaştırma kullanılır. i ve I harflerine özel test yazılır.
Türkçede yanlış çalışmaktadır	Karşılaştırmadan önce boşluklar kırılır ve Unicode biçimi eşitlenir.
Aynı görünen iki metin eşit çıkmamaktadır	Çıktıya yazılırken kaçış (<i>escaping</i>) uygulanır.
Kullanıcı metni sayfayı bozmaktadır	

Zaman ve Tarih

İşaret	Müdahale
Saatler bir yerde kaymaktadır	Her şey UTC olarak saklanır; yalnızca gösterilirken yerel saate çevrilir.
Test belirli günlerde başarısız olmaktadır	Kodda saat doğrudan okunmaz, dışarıdan verilir.
İki olayın sırası karışmaktadır	Sıralama için saate güvenilmez, sıra numarası kullanılır.

Arama ve Listeleme

İşaret	Müdahale
Liste büyüdükçe sayfa açılmamaktadır	Sayfalama eklenir. Tamamı çekilmez.
Sayfalar arasında kayıt atlanmakta ya da tekrar etmektedir	Sıra numarasına göre değil, son görülen kayda göre sayfalanır.
Sıralama her seferinde farklı çıkmaktadır	Sıralamaya benzersiz bir alan eklenir.
Arama yavaşır	Aranan alana uygun indeks eklenir ya da ayrı bir arama yapısı kurulur.

İçe ve Dışa Aktarma

İşaret	Müdahale
Toplu yükleme yarıda kesilmiştir, nerede kaldığı belli değildir	Satır satır durum tutulur. Kaldığı yerden devam edebilir hâle getirilir.
Tek bir bozuk satır tüm yüklemeyi iptal etmektedir	Bozuk satırlar ayrılır, geri kalanı işlenir, sonda rapor verilir.
Dışa aktarma zaman aşımına düşmektedir	Arka planda üretilir; hazır olduğunda indirme bağlantısı verilir.
Aynı dosya iki kez yüklenmiş, kayıtlar çiftlenmiştir	Her satıra doğal bir anahtar belirlenir; varsa güncellenir.

Katalog: Çalışma Zamanı

Zaman aşımı, eşzamanlılık sınırı ve arızanın yayılmasını engelleyen bölmeler, üretim sistemlerinde kararlılığın temel örüntüleridir (Nygard, 2007).

Çalışma Zamanı

İşaret	Müdahale
Kuyruk sürekli büyümektedir	Kuyruğa üst sınır konur. Dolduğunda yeni iş reddedilir ya da bekletilir.
Bir istek sonsuza kadar beklemektedir	Her dış çağrıya zaman aşımı konur.
Yük arttığında her şey birlikte çökmektedir	Eşzamanlılık sınırlanır. Aynı anda en fazla N iş çalışır.
Yeniden deneme sistemi boğmaktadır	Deneme sayısı sınırlanır, denemeler arasındaki süre artırılır.

Eşzamanlılık

İşaret	Müdahale
Hata yalnızca ara sıra ve yük altında çıkmaktadır	Paylaşılan veri bulunur. Ya paylaşım kaldırılır ya da erişim tek kapıdan geçirilir.
İki işlem aynı kaydı güncelleyip birbirini ezmektedir	Sürüm numarası eklenir; eski sürümle gelen yazma reddedilir.
Kilitler çoğalmış, sistem ara sıra donmaktadır	Kilitler hep aynı sırayla alınır. Kilit tutma süresi kısaltılır.
Arka plan işi kimsenin haberi olmadan ölmektedir	Her arka plan işinin hatası yakalanır, kaydedilir, iş yeniden başlatılır.

Kaynak Yönetimi

İşaret	Müdahale
Açık bağlantı sayısı zamanla artmaktadır	Açılan her şeyin kapandığı yer bulunur. Açma ve kapama aynı blokta yazılır.
Bağlantı havuzu tükenmektedir	Havuza sınır konur, bekleme süresi verilir, bağlantıyı uzun tutan sorgu bulunur.
Arka plan işleri birikmekte ve bitmemektedir	Her işe iptal yolu ve süre sınırı verilir.
Kapanırken yarım iş kalmaktadır	Kapanma sinyalinde yeni iş alımı kesilir, eldeki bitirilir, sonra çıkılır.

Önbellek

İşaret	Müdahale
Kullanıcı eski veri görmektedir	Her önbellek kaydına süre verilir. Veri değiştiğinde ilgili kayıt silinir.
Önbellek boşaldığında sistem çökmektedir	Sistem önbelleksiz de ayakta kalabilmelidir. Kalamıyorsa önbellek gizli bir bağımlılıktır.
Neyin önbellekte olduğu bilinmemektedir	Önbellek tek bir yerden yönetilir, dağınık kullanılmaz.

Mesajlaşma ve Kuyruk

İşaret	Müdahale
Aynı mesaj iki kez işlenmiştir	İşleyici tekrara dayanıklı yapılır. İşlenen mesajın kimliği kaydedilir.
Bozuk bir mesaj kuyruğu tıkamaktadır	Belirli sayıda denemeden sonra ayrı bir kuyruğa alınır.
Mesajların sırası bozulmaktadır	Sıraya bağımlı işler aynı anahtarla aynı işleyiciye gönderilir.
Mesaj biçimi değiştiğinde eski işleyici çökmektedir	Mesaja sürüm alanı eklenir. Bilinmeyen alanlar yok sayılır.

Gerçek Zamanlı Bağlantı

İşaret	Müdahale
Bağlantı koptuğunda istemci eski veride kalmaktadır	Yeniden bağlanıldığında tam durum baştan çekilir.
Bağlantı kopmuştur ama kimse fark etmemektedir	Düzenli yoklama mesajı gönderilir; cevap gelmezse bağlantı kapatılır.
Herkese her olay gönderilmektedir	Abonelik eklenir. İstemci yalnızca ilgilendiği konuyu alır.
Yavaş istemci sunucuda belleği şişirmektedir	Gönderim tamponuna sınır konur. Sınırı aşan istemcinin bağlantısı kesilir.

Zamanlanmış İşler

İşaret	Müdahale
İş çalışmamıştır ve kimse fark etmemiştir	Her çalışmanın başı ve sonu kaydedilir. Beklenen saatte bitmezse alarm üretilir.
Aynı iş iki sunucuda birden çalışmıştır	İş başlamadan kilit alır. Kilidi alamayan çıkar.
İş bir öncekisi bitmeden yeniden başlamıştır	Çakışma yasaklanır ya da iş kısaltılır.
Kaçan bir çalışmayı elle tekrarlamak zordur	İş tarih parametresiyle çalışır hâle getirilir.

Dosya ve Depolama

İşaret	Müdahale
Büyük dosya belleği doldurmaktadır	Tamamı belleğe alınmaz; akış hâlinde okunur ve yazılır.
Yarım yüklenen dosyalar birikmektedir	Geçici alana yüklenir, tamamlanınca taşınır. Eski geçiciler düzenli silinir.
Dosya adı çakışmakta ya da tuhaf karakter içermektedir	Kullanıcının verdiği ad saklama anahtarı yapılmaz. Sistem kendi kimliğini üretir.
Disk dolmaktadır	Saklama süresi belirlenir, eskiler otomatik silinir. Doluluk için alarm kurulur.

Medya İşleme

İşaret	Müdahale
İşleme uzun sürmekte, istek zaman aşımına düşmektedir	İş arka plana alınır. İsteğe hemen iş kimliği dönülür.
Bozuk bir dosya işleyiciyi çökertmektedir	İşleme ayrı süreçte çalıştırılır; süre ve bellek sınırı konur.
Aynı dosya tekrar tekrar işlenmektedir	İçerik özeti saklanır; aynı özet gelirse eski sonuç dönülür.
İlerleme bilinmemektedir	İş adımlara bölünür; her adım bittiğinde durum kaydedilir.

Performans

Ölçmeden yapılan iyileştirmenin çoğu zaman yanlış yere yöneldiği Knuth (1974) tarafından belirtilmiştir.

İşaret	Müdahale
“Yavaş” hissi vardır ama yeri belli değildir	Önce ölçülür. Ölçmeden değiştirilmez.
Bir döngünün içinde veritabanı sorgusu vardır	Sorgu döngünün dışına alınır, veriler toplu çekilir.
Bellek zamanla büyümektedir	Sınırsız büyüyen yapılar bulunur: önbellek, liste, dinleyici. Hepsine üst sınır konur.
Hızlandırma kodu okunmaz hâle getirmiştir	Kazanç ölçülür. Küçükse geri alınır.

Harici Bağımlılıklar

İşaret	Müdahale
Bir kütüphane güncellemesi bir şeyi bozmuştur	Sürümler kilitlenir. Güncelleme tek tek ve ayrı <i>commit</i> 'lerle yapılır.
Dış servis yavaşladığında sistem de yavaşlamaktadır	Zaman aşımı, önbellek ve yedek davranış eklenir.
Küçük bir iş için büyük bir kütüphane eklenecektir	İş elle yazılır ya da daha küçük bir kütüphane bulunur.

Bildirim

İşaret	Müdahale
Kullanıcıya aynı bildirim birkaç kez gitmektedir	Gönderilenler kaydedilir, tekrarı engellenir.
Bildirimler çoktur, kullanıcı kapatmaktadır	Birleştirilir. Önemsizler özet hâline getirilir.
Gönderim başarısız olduğunda kaybolmaktadır	Kuyruğa alınır, sınırlı sayıda yeniden denenir.

Katalog: Güvenilirlik ve Güvenlik

Hata Yönetimi

İşaret	Müdahale
Hata yutulmakta, hiçbir yerde görünmemektedir	Boş catch bırakılmaz. Hata ya işlenir ya yukarı iletilir.
Hata mesajından sorunun yeri anlaşılmamaktadır	Mesaja bağlam eklenir: hangi kayıt, hangi adım, hangi girdi.
Bir parça çöktüğünde her şey durmaktadır	O parça ayrılır. Çöktüğünde geri kalan çalışmaya devam eder.
İşlem yarıda kaldığında veri bozuk kalmaktadır	İşlem transaction içine alınır ya da tekrar çalıştırılabilir yapılır.

Test

İşaret	Müdahale
Hata canlı ortamda bulunmuştur	Önce o hatayı yakalayan test yazılır, sonra düzeltilir.
Testler bazen geçmekte, bazen kalmaktadır	Test hemen düzeltilir ya da silinir. Güvenilmeyen test, testsizlikten kötüdür.
Test paketi çok yavaştır ve çalıştırılmamaktadır	Hızlı testler ayrılır. Her kayıta hızlılar, yayından önce hepsi çalışır.
Test, kod her değiştiğinde kırılmaktadır	Test iç yapıya bağlanmıştır. Uygulama değil, davranış test edilir.

Kayıt Tutma

İşaret	Müdahale
Sorun çıkmıştır, kayıtlarda iz yoktur	Her isteğe bir kimlik verilir ve tüm satırlara yazılır.
Kayıtlar çoktur, aranan bulunamamaktadır	Seviyeler ayrılır. Normal akışta yalnızca önemli olaylar yazılır.
Kayıt satırı serbest metindir, aranmamaktadır	Yapılı biçime geçilir: alan adı ve değer.
Kayıtlarda parola ya da kişisel veri görünmektedir	Yazılmadan önce maskelenir.

İzleme

İşaret	Müdahale
Alarmlar okunmadan kapatılmaktadır	Alarm sayısı azaltılır. Yalnızca harekete geçilmesi gerekenler kalır.
Sorun çıktığında nereye bakılacağı bilinmemektedir	Beş temel sayı seçilir: istek, gecikme, hata, kuyruk, kaynak. Önce onlara bakılır.

Kurtarma

İşaret	Müdahale
Yedek vardır ama hiç geri yüklenmemiştir	Geri yükleme denenir. Denenmemiş yedek, yedek sayılmaz.
Arıza sırasında ne yapılacağı bilinmemektedir	Adımlar önceden yazılır; arıza anında düşünülmez.
Aynı arıza ikinci kez yaşanmıştır	Kök neden bulunur ve ortadan kaldırılır; belirti yamanmaz.

Güvenlik

İşaret	Müdahale
Kullanıcıdan gelen veri doğrudan sorguya girmektedir	Parametrelili sorgu kullanılır.
Yetki kontrolü yalnızca arayüzdedir	Sunucuda da kontrol edilir. Arayüze güvenilmez.
Herkes her şeye erişebilmektedir	Varsayılan “yasak” yapılır; gereken tek tek açılır.
Bir anahtar çok yerde kullanılmaktadır	Her servise ayrı anahtar verilir, yetkisi daraltılır.

Oturum ve Kimlik

İşaret	Müdahale
Kullanıcı durduk yere oturumdan düşmektedir	Belirteç (<i>token</i>) süresi ve yenileme akışı kontrol edilir.
Çıkış yapan kullanıcının belirteci hâlâ geçerlidir	Belirteçler kısa ömürlü yapılır ya da iptal listesi tutulur.
Rol kontrolleri kodun her yerine dağılmıştır	Tek bir yetki katmanında toplanır.
Parola sıfırlama bağlantısı tekrar kullanılabilir	Tek kullanımlık yapılır ve süre verilir.

Çok Kullanıcı Sistem

İşaret	Müdahale
Bir müşterinin verisi başkasına görünmüştür	Müşteri kimliği her sorguda zorunlu kılınır. Tek tek hatırlamaya bırakılmaz.
Bir müşterinin yükü herkesi yavaşlatmaktadır	Müşteri başına sınır konur.
Müşteriye özel if blokları çoğalmıştır	Farklar ayara taşınır, koddan çıkarılır.

Yasal ve Gizlilik

İşaret	Müdahale
Kişisel veri gereğinden uzun saklanmaktadır	Saklama süresi belirlenir, süresi dolan silinir.
Kimin neye baktığı bilinmemektedir	Hassas veriye erişim kaydedilir.

Katalog: Değişiklik ve Yayın

Küçük ve sık yayının büyük ve seyrek yayından daha az risk taşıdığı, sürekli teslim yaklaşımının temel savıdır (Humble ve Farley, 2010).

Değişiklik

İşaret	Müdahale
Değişiklik 300–400 satırı geçmiştir	<i>Commit</i> bölünür. Her <i>commit</i> tek bir şeyi değiştirir.
<i>Commit</i> mesajında “ve” kullanılmaktadır	İki ayrı <i>commit</i> yapılır.
Değişikliği geri almak korkutmaktadır	Daha küçük adımlarla ilerlenir; her adımda test çalıştırılır.
Yeniden düzenleme ile yeni özellik aynı <i>commit</i> ’tedir	Ayrılır. Önce düzenlenir, sonra eklenir.

Sürüm Kontrolü

İşaret	Müdahale
Dal haftalardır açıktır, birleştirmesi zordur	Dal kısa tutulur. Her gün ana dala yetiştir, birkaç günde birleştirilir.
Birleştirme çakışmaları sürekli aynı dosyadadır	O dosya çok iş yapmaktadır. Bölünür.
Hangi <i>commit</i> ’in neyi bozduğu bulunamamaktadır	<code>git bisect</code> kullanılır. Bunun için her <i>commit</i> tek başına çalışır olmalıdır.
Yarım iş ana dala girmek zorundadır	Özellik bayrağının arkasına konur, kapalı yayınlanır.

Kod İnceleme

İşaret	Müdahale
İnceleme günlerce beklemektedir	Değişiklik küçültülür. Küçük değişiklik hızlı incelenir.
İncelemede aynı yorumlar tekrar etmektedir	Kural otomatik denetime çevrilir.
İnceleyen kişi ne yapıldığını anlamamaktadır	Açıklamaya neyin, neden yapıldığı ve nasıl test edildiği yazılır.

Derleme ve Otomasyon

İşaret	Müdahale
Derleme uzamıştır, beklerken başka işe geçilmektedir	Süre ölçülür, en yavaş adım bulunur, önbellege alınır ya da paralel çalıştırılır.
Otomatik denetim kırmızıdır ve kimse bakmamaktadır	Kırmızıyken yeni iş birleştirilmez. Önce yeşile çevrilir.
Bir adım her seferinde elle yapılmaktadır	Üçüncü tekrarda betiğe çevrilir.
Yayın adımları bir kişinin aklındadır	Adımlar yazılır, sonra otomatige bağlanır.

Yayın

İşaret	Müdahale
Sürümden sonra bir şey bozulmuştur, sebebi belli değildir	Sürüm küçültülür. Tek seferde daha az değişiklik yayınlanır.
Yayınlamaktan çekinilmektedir	Daha sık yayınlanır; geri alma yolu önceden hazırlanır.

Özellik Bayrakları

Özellik bayrağı (*feature flag*), bir kod yolunu yeniden yayın yapmadan açıp kapatmaya yarayan ayardır.

İşaret	Müdahale
Bayrak sayısı artmıştır, hangisinin açık olduğu belli değildir	Her bayrağa sahip ve son tarih verilir. Süresi geçen kaldırılır.
Bayrak kalıcı hâle gelmiştir	Ya avara çevrilir ya da kod tek yola indirilir.
İki bayrağın birleşimi hiç test edilmemiştir	Bayraklar birbirinden bağımsız tutulur. Bağımlıysa tek bayrağa indirilir.

Yapılandırma ve Ortam

İşaret	Müdahale
“Bende çalışıyor” durumu ortaya çıkmıştır	Ortam sabitlenir: sürümler kilitlenir, kurulum tek komuta indirilir.
Yapılandırma seçenekleri çoğalmıştır, kimse hepsini bilmemektedir	Kullanılmayanlar silinir, geri kalanına makul varsayılan verilir.
Gizli bilgi kodun içindedir	Hemen çıkarılır, ortam değişkenine taşınır, anahtar yenilenir.

Maliyet

İşaret			Müdahale
Fatura	beklenmedik	biçimde	Harcama servise göre ayrılır, en büyük kalem bulunur.
Kullanılmayan		kaynaklar	Her kaynağa sahip etiketi konur. Sahipsiz olan kapatılır.
çalışmaktadır			
Ölçekleme sınırsızdır			Üst sınır konur, bütçe alarmı kurulur.

Katalog: Kodun Yaşam Döngüsü

Teknik Borç

İşaret	Müdahale
“Sonra düzeltirim” notları birikmiştir	Her işin başında ilgili yerden bir tanesi temizlenir.
Kullanılmayan kod durmaktadır	Silinir. Sürüm kontrolünde zaten vardır.
Bir bölgede hatalar toplanmaktadır	Yamanmaz, yeniden yazılır. Önce testle çevrilir.
Geçici çözüm kalıcı hâle gelmiştir	Ya düzgün hâle getirilir ya da neden böyle kaldığı yazılır.

Eski Kod

Anlaşılmayan kodun mevcut davranışının değişiklikten önce testle sabitlenmesi yöntemi Feathers (2004) tarafından ayrıntılı biçimde anlatılmıştır.

İşaret	Müdahale
Dokunulması gereken kod anlaşılmamaktadır	Değiştirmeden önce mevcut davranış testle sabitlenir.
Kodun neden böyle yazıldığı belli değildir	Geçmişine bakılır. Sebep bulunamazsa silmeden önce etkisi ölçülür.
Yeniden yazma isteği vardır	Parça parça değiştirilir. Eski ve yeni bir süre yan yana çalışır.
Tuhaf ve gereksiz görünen bir koşul vardır	Kaldırmadan önce o koşulu tetikleyen durum aranır.

Taşıma Projeleri

İşaret	Müdahale
Büyük bir geçiş tek seferde planlanmaktadır	Aşamalara bölünür. Her aşamanın sonunda sistem çalışır durumdadır.
Geçiş sırasında veri iki yeredir ve uyuşmamaktadır	Tek kaynak belirlenir. Diğeri ondan beslenir, farklar düzenli karşılaştırılır.
Geri dönüş yolu yoktur	Geçişten önce geri dönüş denenir.
Eski sistem kapatılamamaktadır	Kalan kullanım listelenir, her birine tarih verilir.

Prototip

İşaret	Müdahale
Deneme kodu ürüne dönüşmek üzeredir	Karar verilir: ya düzgün biçimde yeniden yazılır ya da eksikleri listelenip kapatılır.
Bir fikrin işe yarayıp yaramayacağı belirsizdir	En riskli kısım önce denenir, geri kalanı sonra yazılır.
Deneme uzamıştır	Süre sınırı konur. Dolduğunda sonuç yazılır ve karar verilir.

Yapay Zekâ ile Kod Yazma

İşaret	Müdahale
Üretilen kod anlaşılmadan kabul edilmektedir	Satır satır okunur. Anlaşılmayan kod birleştirilmez.
Tek istekte çok büyük değişiklik gelmiştir	İstek küçültülür. Her adım ayrı incelenir.
Model aynı hatayı tekrarlamaktadır	Kural açıkça yazılır ve kalıcı talimata eklenir.
Üretilen kod mevcut yapıya uymamaktadır	Örnek bir dosya gösterilir, ona benzetilmesi istenir.
Çalışıyor görünmektedir ama test yoktur	Önce test, sonra kod yazdırılır.

Katalog: Arayüz

Arayüz Durumu

İşaret	Müdahale
Ekran sunucudaki veriyle uyuşmamaktadır	Sunucu verisi tek yerde tutulur; ekran oradan okur.
Bir bileşen çok fazla girdi almaktadır	Bölünür ya da ilgili girdiler bir nesnede toplanır.
Eski isteğin yanıtı yenisinin üzerine yazmaktadır	Yeni istek gönderildiğinde eskisi iptal edilir.
Yükleniyor, hata ve boş durumları unutulmaktadır	Veri çeken her ekranda dört durum açıkça ele alınır: yükleniyor, hata, boş, dolu.

Uluslararasılaştırma

İşaret	Müdahale
Metinler kodun içine gömülüdür	Çeviri dosyasına taşınır.
Çeviride cümle parçaları birleştirilmektedir	Tam cümle kullanılır; değişkenler içine yerleştirilir.
Tarih ve sayı biçimi ülkeye göre yanlıştır	Biçimlendirme dil ayarına bağlanır, elle yazılmaz.

Erişilebilirlik

İşaret	Müdahale
Arayüz yalnızca fareyle kullanılabilir	Her işlem klavyeyle de yapılabilir hâle getirilir.
Bilgi yalnızca renkle verilmektedir	Yanına metin ya da simge eklenir.

Mobil ve Zayıf Ağ

İşaret	Müdahale
Bağlantı yavaşken ekran boş kalmaktadır	Önce eldeki veri gösterilir; yenisi geldiğinde güncellenir.
İstek yarıda kesildiğinde işlem belirsiz kalmaktadır	İstek tekrar gönderilebilir yapılır; sonuç sunucudan doğrulanır.

Katalog: İş ve Kişisel Çalışma

İş Akışı

Aynı anda yürüyen iş sayısının sınırlanması, Kanban yönteminin temel uygulamasıdır (Anderson, 2010).

İşaret	Müdahale
Aynı anda üçten fazla iş açıktır	Yeni işe başlanmaz. Biri bitirilir.
İş tahmin edilenin iki katına çıkmıştır	Durulur. Kapsam küçültülür, kalan kısım ayrı iş yapılır.
Yapılan iş bir cümleyle anlatılamamaktadır	Yazmaya ara verilir. Önce problem yazılır, sonra koda dönülür.

Gereksinim

İşaret	Müdahale
İstenen şey belirsizdir	Kod yazmadan önce bir örnek girdi ve beklenen çıktı yazıya dökülür.
İş bitmiştir ama istenen bu değildir	Bir sonrakinde erken gösterilir. Tamamı beklenmez, yarım hâli gösterilir.
İstek iş sırasında sürekli büyümektedir	Yeni istekler ayrı listeye yazılır, mevcut iş bitirilir.
“Belki gerekir” diye bir şey eklenmektedir	Eklenmez. Gerektiğinde eklenir.

Tahmin ve Planlama

İşaret	Müdahale
Tahminler tutmamaktadır	Geçmiş tahminler ve gerçekleşen kaydedilir; fark bir sonraki tahmine eklenir.
İş büyüktür, süre verilememektedir	Bir günden kısa parçalara bölünür. Bölünemeyen parça bilinmeyendir; önce o araştırılır.
Plan ilk gün bozulmuştur	Plan güncellenir. Eskisine uymaya çalışılmaz.
Her şey acildir	Sıralanır. Aynı anda yalnızca biri birinci olabilir.

Karar

İşaret	Müdahale
İki seçenek arasında kalınmıştır, fark küçüktür	Geri alması kolay olan seçilir ve ilerlenir.
Karar geri alnamaz niteliktedir	Yavaşlanır. Küçük bir deneme yapılır, sonra karar verilir.
Aynı tartışma üçüncü kez açılmaktadır	Karar ve gerekçesi yazılır, bir yere kaydedilir.

İletişim

İşaret	Müdahale
Bir şey beklenirken takılınmıştır	Hemen sorulur; beklerken başka işe geçilir.
Yazışma uzamıştır ve sonuç çıkmamaktadır	Kısa bir görüşme yapılır, karar yazıya dökülür.
Toplantıdan çıkıldığında kimin ne yapacağı belli değildir	Toplantı bitmeden iş, sahip ve tarih yazılır.
Kötü haber vardır	Erken söylenir. Seçenekleriyle birlikte söylenir.
Aynı durum farklı kişilere tekrar anlatılmaktadır	Bir kez yazılır, herkese aynı metin gönderilir.

Dokümantasyon

İşaret	Müdahale
Aynı soru ikinci kez cevaplanmaktadır	Cevap yazılır; bir sonrakinde bağlantısı verilir.
Doküman kodla uyumamaktadır	Doküman kodun yanına taşınır; birlikte değişirler.
Kurulum her seferinde uğraştırmaktadır	Adımlar betiğe çevrilir.
Bir kararın nedeni unutulmuştur	Kararlar kısa notlarla kaydedilir: ne seçildi, neden, neye karşı.

Öğrenme

İşaret	Müdahale
Bir araç ezberden kullanılmakta, bozulduğunda çaresiz kalınmaktadır	Nasıl çalıştığı bir kez oturulup okunur.
Aynı şey her seferinde aranıp bulunmaktadır	Not alınır; kişinin kendi sözcükleriyle yazılır.
Bir hata çok uğraştırmıştır	Çözüldüğünde nedeni ve çözümü kaydedilir.
Yeni bir teknoloji projeye sokulmak istenmektedir	Önce küçük ve atılabilir bir denemede kullanılır.

Kişisel Çalışma

İşaret	Müdahale
Aynı hata 30 dakikadır kovalanmaktadır	Durulur. Problem yazıya dökülür ya da ara verilir.
Tahminle değişiklik yapıp denenmektedir	Bırakılır. Bir varsayım kurulur ve tek bir deneyle sınanır.
Akılda çok şey tutulmaya çalışılmaktadır	Kâğıda yazılır. Akılda yalnızca o anki adım kalır.
Gün sonunda ne yapıldığı hatırlanmamaktadır	Her işin başına ve sonuna tek satır not düşülür.
Yorgunken büyük bir değişikliğe girilmektedir	Ertelenir. Yorgunken yalnızca küçük ve geri alınabilir işler yapılır.

Odak

İşaret	Müdahale
Çalışma sürekli bölünmektedir	Bildirimler kapatılır; belirli saatlerde toplu bakılır.
Bir işten diğerine atlanmaktadır	Geçmeden önce nerede kalındığı tek satırla yazılır.
Başlamak zor gelmektedir	İlk adım beş dakikalık bir işe indirilir.
Günün sonunda önemli iş yapılmamıştır	Güne en önemli işle başlanır, diğerleri sonraya bırakılır.

Bitirme

İşaret	Müdahale
İş “neredeyse bitti” durumunda beklemektedir	Kalanlar listelenir. Liste uzunsa iş bitmemiştir; yeniden planlanır.
Çalışmaktadır ama emin olunamamaktadır	Bozulabilecek üç durum yazılır ve denenir.
İş bitmiştir ama izlemesi yoktur	Çalıştığının nereden anlaşılacağı belirlenir. Yoksa bir ölçüm eklenir.
Yayından sonra hiç bakılmamıştır	Ertesi gün bir kez kontrol edilir.

Müdahaleden Sonra ve Genelleme Kalıbı

Her Müdahaleden Sonra

İşaret	Müdahale
Bir sınır sıkılaştırılmıştır	Karmaşıklığın nereye gittiğine bakılır. Kaybolmadıysa bir üst seviyededir: dosyadan modüle, modülden bağımlılığa, bağımlılıktan duruma.
Sınır işi zorlaştırmaktadır ama den- gesizlik yoktur	Sınır gevşetilir. Sayılar başlangıç noktasıdır; gerekçesi kalmayan sınır kaldırılır.

Katalogu Uzatmak İçin Kalıp

Katalogdaki ana alanlar tamamlanmıştır; bundan sonra eklenecek maddeler ya dar alanlara özgüdür ya da mevcut maddelerin tekrarıdır. Yeni bir durumla karşılaşıldığında dört adım yeterlidir:

1. **İşaret yazılır.** Neyin kestirilemediği ya da neyin akılda tutulamadığı belirlenir.
2. **Seviye bulunur.** Sorunun kodda mı, veride mi, durumda mı, çalışma zamanında mı, iş akışında mı olduğu saptanır.
3. **O seviyede tek bir sınır konur.** Sınır şu türlerden biridir: boyut, sayı, süre, izin verilen değerler ya da izin verilen geçişler.
4. **Sonuca bakılır.** Karmaşıklığın kaybolup kaybolmadığı ya da başka bir seviyeye geçip geçmediği kontrol edilir.

Katalogdaki bütün maddeler bu kalıbın farklı yerlere uygulanmış hâlidir.

Sonuç

Disiplin ile özgürlük arasındaki karşıtlık, özgürlük “engelin yokluğu” olarak tanımlandığında geçerlidir. Özgürlük “fiilen yapılabilenlerin genişliği” olarak tanımlandığında karşıtlık ortadan kalkar ve disiplin, özgürlüğü üreten süreç hâline gelir. Bunun pratik karşılığı, bir alanda seçenek azlığı hissedildiğinde ilk bakılacak yerin kuralların çokluğu değil, sınır bilgisinin azlığı olmasıdır.

İkinci çıkarım, kısıtların bir tasarım aracı olarak ele alınabileceğidir. Bir problemin kısıtlarını tek tek yazmak, durumlarını ve geçişlerini çıkarmak, kısıtlar arasındaki ilişkileri belirlemek, çözüm üretmenin ön hazırlığı değil, çözümün kendisinin büyük bir bölümüdür.

Üçüncü çıkarım, ustalığın dışarıdan yanlış okunabileceğidir. Usta dışarıdan daha fazla kurala uyan kişi olarak görünür; içeriden bakıldığında ise daha fazla seçeneğe sahip olan kişidir.

Dördüncü çıkarım, sınırın aramayı da yönettiğidir. Kısıtlar yalnızca neyin mümkün olduğunu göstermez, hangi seçeneklere hiç bakılmaması gerektiğini de gösterir. Satrançta ustalığı ayıran şey hesaplama miktarı değil, aday hamlelerin kalitesidir.

Beşinci çıkarım, temsil seçiminin bir tasarım kararı olduğudur. Bir sisteme hangi beş sayıyla bakılacağına karar vermek, o sistem hakkında hangi kararların verilebileceğini de belirler.

Altıncı çıkarım, sınırın kasıtlı olarak konabileceğidir. Kavramsal bölümde sınır çoğunlukla dışarıdan verilen bir şeydir: yerçekimi, malzeme, ağ. Katalogda ise sınır, kararlılığı geri kazanmak için içeriden konan bir şeydir: 200 satır, üç açık iş, tek durum alanı. İki tür aynı işi görür; hareket alanını akılda tutulabilir boyuta indirir ve o alanın içinde serbestçe çalışmayı mümkün kılar.

Çerçevenin sınırlılıkları şunlardır:

- Her sınır üretken değildir. Fiziksel sınırlar ile keyfi ya da bürokratik kurallar aynı türden değildir. Çerçeve, bu iki tür sınırı ayırt etmek için bir ölçüt vermez.
- Doğal sınırlar ile uzlaşımsal sınırlar farklı davranır. Yerçekimi değiştirilemez; balenin teknik kuralları ya da bir mühendislik standardı ise insanlar tarafından konmuştur ve değiştirilebilir.
- Uzmanlık katılaşmaya da yol açabilir. Luchins (1942), belirli bir çözüm yöntemine alışan kişilerin daha basit bir yöntem mümkün olduğunda bile alıştıkları yöntemi sürdürdüğünü göstermiştir.
- Örnekler başarılı ustalık durumlarından seçilmiştir. Disiplinin özgürlüğe dönüşmediği, yalnızca tekrara dönüştüğü durumlar ele alınmamıştır.
- Yoğun temsil, atılan görünmez kılar. Panelde yer almayan bir ölçüm, sorun tam orada olduğunda fark edilmez. Ayrıca yoğunluk okuyana bağlıdır: `GET /users/42` ifadesi HTTP bilen biri için yoğun, bilmeyen biri için anlamsızdır.

- Katalog deneyime dayalı bir kurallar listesidir, ölçümle doğrulanmış değildir. İşaretlerin çoğu öznel bir hisse dayanır; “dokunmaktan çekinmek” kişiden kişiye farklı eşikte ortaya çıkar. Sayısal eşikler bağlama göre değişir.
- Çok fazla sınır da bir karmaşıklık kaynağıdır. Katalogdaki bütün maddeleri aynı anda uygulamak, akılda tutulması gereken kural sayısını artırır ve çıkış noktasındaki sorunu yeniden üretir.

Açık kalan sorular da vardır. Bir sınırın içinde çalışmak ile o sınırı değiştirmeye girişmek arasındaki karar neye göre verilir? Serbestlik derecesi arttıkça kontrolün zorlaştığı bilindiğine göre, belirli bir beceri düzeyi için uygun serbestlik miktarı nasıl belirlenir? Bir sınırın gerçek mi yoksa yalnızca alışkanlık mı olduğu nasıl sınanır? Karmaşıklığın yer değiştirdiği ile gerçekten azaldığı durumlar birbirinden nasıl ayırt edilir?

Kaynaklar

- Anderson, D. J. (2010). *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press.
- Berlin, I. (1958). *Two Concepts of Liberty*. Clarendon Press.
- Box, G. E. P. (1976). Science and statistics. *Journal of the American Statistical Association*, 71(356).
- Boyd, S. ve Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.
- Brooks, F. P. (1987). No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4).
- Chase, W. G. ve Simon, H. A. (1973). Perception in chess. *Cognitive Psychology*, 4(1).
- de Groot, A. D. (1965). *Thought and Choice in Chess*. Mouton.
- Dreyfus, H. L. ve Dreyfus, S. E. (1986). *Mind over Machine: The Power of Human Intuition and Expertise in the Era of the Computer*. Free Press.
- Feathers, M. (2004). *Working Effectively with Legacy Code*. Prentice Hall.
- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doktora tezi, University of California, Irvine.
- Fowler, M. (1999). *Refactoring: Improving the Design of Existing Code*. Addison-Wesley.
- Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1).
- Humble, J. ve Farley, D. (2010). *Continuous Delivery*. Addison-Wesley.

- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Knuth, D. E. (1974). Structured programming with go to statements. *ACM Computing Surveys*, 6(4).
- Korzybski, A. (1933). *Science and Sanity*. International Non-Aristotelian Library.
- Kruger, J. ve Dunning, D. (1999). Unskilled and unaware of it: How difficulties in recognizing one's own incompetence lead to inflated self-assessments. *Journal of Personality and Social Psychology*, 77(6).
- Luchins, A. S. (1942). Mechanization in problem solving: The effect of Einstellung. *Psychological Monographs*, 54(6).
- Miller, G. A. (1956). The magical number seven, plus or minus two. *Psychological Review*, 63(2).
- Nygard, M. T. (2007). *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf.
- Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12).
- Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27.
- Shannon, C. E. (1950). Programming a computer for playing chess. *Philosophical Magazine*, 41(314).
- Simon, H. A. (1969). *The Sciences of the Artificial*. MIT Press.
- Stravinsky, I. (1947). *Poetics of Music in the Form of Six Lessons*. Harvard University Press.
- Tufte, E. R. (1983). *The Visual Display of Quantitative Information*. Graphics Press.